

Bomblab 解题报告

姓名：罗呈语

学号：2024201548

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	1	1	1	1	1	1

scoreboard 截图：

2024201548	0	0	0	0	0	0	0	0	0	0	0	0	0	0	7
------------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Phase 1：字符串比较

通过阅读汇编代码

```
0000000000001435 <phase_1>:
1435: 48 83 ec 08          sub    $0x8,%rsp
1439: 48 8d 35 40 1d 00 00 lea     0x1d40(%rip),%rsi      # 3180 <_IO_stdin_used+0x180>
1440: e8 3f 08 00 00      call   1c84 <strings_not_equal>
1445: 85 c0              test   %eax,%eax
1447: 75 05             jne     144e <phase_1+0x19>
1449: 48 83 c4 08       add     $0x8,%rsp
144d: c3              ret
144e: e8 96 0a 00 00      call   1ee9 <explode_bomb>
1453: eb f4             jmp     1449 <phase_1+0x14>
```

可以看出，程序调用了 `strings_not_equal` 函数来比较输入字符串与某个预设字符串。如果不相等，则调用 `explode_bomb` 函数引爆炸弹。

通过使用gdb指令

```
(gdb) x/s 0x0000555555557180
```

可以查看到预设字符串为：

Abidel Rid wezha ray frockeme zaridew encor ed xas. An betdariuzis konsis dlle.

因此，Phase 1 的正确输入为：

Abidel Rid wezha ray frockeme zaridew encor ed xas. An betdariuzis konsis dlle.

Phase 2：矩阵乘法

通过阅读汇编代码，可以看出，程序首先使用 `sscanf` 函数从输入中读取了四个整数，并将它们存储在栈上。然后，程序进行了矩阵乘法运算，使用了两个预设的矩阵 `matA` 和 `matB`。最后，程序将结果与输入的四四个整数进行比较，如果不匹配则引爆炸弹。

其中35f1 <array.0+0x391> 对应格式字符串 `%d %d %d %d`，用于读取四个整数。

使用gdb指令

```
(gdb) x/6wd 0x000055555555a130
0x55555555a130 <matA.2>:      364      849      599      914
```

```
0x55555555a140 <matA.2+16>:      580      200

(gdb) x/6wd 0x000055555555a110
0x55555555a110 <matB.1>:      386      329      964      119
0x55555555a120 <matB.1+16>:    116      911
```

以及汇编代码中

```
148e:  48 8d 3d 9b 4c 00 00    lea    0x4c9b(%rip),%rdi      # 6130 <matA.2>
14bb:  48 8d 35 4e 4c 00 00    lea    0x4c4e(%rip),%rsi      # 6110 <matB.1>
14d8:  8b 14 87                mov     (%rdi,%rax,4),%edx
14db:  0f af 14 c6             imul   (%rsi,%rax,8),%edx
14df:  01 d1                  add     %edx,%ecx
```

可以看出，矩阵 `matA` 是一个 2 行 3 列的矩阵，矩阵 `matB` 是一个 3 行 2 列的矩阵。程序通过嵌套循环实现了矩阵乘法运算。

于是可以得出矩阵 `matA` 和 `matB` 如下：

```
matA = | 364 849 599 |
        | 914 580 200 |

matB = | 386 329 |
        | 964 119 |
        | 116 911 |
```

通过计算矩阵乘法 `matA * matB`，得到结果矩阵 `matC`：

```
matC = | 1028424 766476 |
        | 935124 551926 |
```

因此，Phase 2 的正确输入为：

1028424 766476 935124 551926

Phase 3: switch 语句

通过阅读汇编代码

```
0000000000001544 <phase_3>:
1572:  83 7c 24 04 00          cmpl   $0x0,0x4(%rsp)
1577:  78 05                  js     157e <phase_3+0x3a>
1579:  e8 6b 09 00 00          call  1ee9 <explode_bomb>
157e:  83 3c 24 07             cmpl   $0x7, (%rsp)
1582:  0f 87 98 00 00 00       ja     1620 <phase_3+0xdc>

1588:  8b 04 24                mov     (%rsp),%eax
158b:  48 8d 15 ae 1c 00 00     lea     0x1cae(%rip),%rdx      # 3240 <_IO_stdin_used+0x240>
1592:  48 63 04 82             movslq  (%rdx,%rax,4),%rax
1596:  48 01 d0                add     %rdx,%rax
1599:  ff e0                  jmp     *%rax

159b:  b8 59 02 00 00          mov     $0x259,%eax
15a0:  2d 49 01 00 00          sub     $0x149,%eax
15a5:  83 c0 4d                add     $0x4d,%eax
15a8:  8d 98 2f fd ff ff       lea     -0x2d1(%rax),%ebx
```

```
15ae:  e8 36 09 00 00      call 1ee9 <explode_bomb>
15b3:  8d 83 d1 02 00 00    lea 0x2d1(%rbx),%eax
15b9:  2d d1 02 00 00      sub $0x2d1,%eax
15be:  05 d1 02 00 00      add $0x2d1,%eax
15c3:  2d d1 02 00 00      sub $0x2d1,%eax

15c8:  83 3c 24 05          cmpl $0x5,(%rsp)
15cc:  7f 06               jg 15d4 <phase_3+0x90>
15ce:  39 44 24 04          cmp %eax,0x4(%rsp)
15d2:  74 05               je 15d9 <phase_3+0x95>
15d4:  e8 10 09 00 00      call 1ee9 <explode_bomb>
15d9:  48 8b 44 24 08      mov 0x8(%rsp),%rax
...
15ee:  c3                  ret

15ef:  b8 00 00 00 00      mov $0x0,%eax
15f4:  eb aa               jmp 15a0 <phase_3+0x5c>
15f6:  b8 00 00 00 00      mov $0x0,%eax
15fb:  eb a8               jmp 15a5 <phase_3+0x61>
15fd:  b8 00 00 00 00      mov $0x0,%eax
1602:  eb a4               jmp 15a8 <phase_3+0x64>
1604:  bb 00 00 00 00      mov $0x0,%ebx
1609:  eb a8               jmp 15b3 <phase_3+0x6f>
160b:  b8 00 00 00 00      mov $0x0,%eax
1610:  eb a7               jmp 15b9 <phase_3+0x75>
1612:  b8 00 00 00 00      mov $0x0,%eax
1617:  eb a5               jmp 15be <phase_3+0x7a>
1619:  b8 00 00 00 00      mov $0x0,%eax
161e:  eb a3               jmp 15c3 <phase_3+0x7f>
1620:  e8 c4 08 00 00      call 1ee9 <explode_bomb>
```

可以看出，程序首先使用 `sscanf` 函数从输入中读取了两个整数 `index` 与 `value`，并检查 `value < 0` 且 `index < 7`。使用 `gdb` 指令我们可以找到跳转表存放处的值如下：

```
0x555555557240: -7333 (Case 0 → 0x159b)
0x555555557244: -7249 (Case 1 → 0x15ef)
0x555555557248: -7242 (Case 2 → 0x15f6)
0x55555555724c: -7235 (Case 3 → 0x15fd)
0x555555557250: -7228 (Case 4 → 0x1604)
0x555555557254: -7221 (Case 5 → 0x160b)
0x555555557258: -7214 (Case 6 → 0x1612)
0x55555555725c: -7207 (Case 7 → 0x1619)
```

接着，程序使用 `index` 作为索引，从一个预设的跳转表中获取相对跳转位置的地址偏移量，并跳转到该地址执行相应的代码段。然后我们发现在程序的必经之路中

```
15c8:  83 3c 24 05          cmpl $0x5,(%rsp)
```

意味着 `index` 必须小于等于 5，并且跳转后的代码块中有一段无法跳过的

```
15ae:  e8 36 09 00 00      call 1ee9 <explode_bomb>
```

因此，我们需要确保在跳转到该地址之后，所以 `index` 的输入必须为4或者5。

通过分析 case 4 与 case 5 的代码，如果我们想让 `value < 0`，输入的 `index` 必须为5，此时 `value` 为-721。

case 5 的代码如下：

```
mov    $0x0,%eax
sub    $0x2d1,%eax
add    $0x2d1,%eax
sub    $0x2d1,%eax
```

因此，Phase 3 的正确输入为：

5 -721

Phase 4：递归函数 func4_1 / func4_2

通过阅读汇编代码

```
00000000000016f0 <phase_4>:
1720:  bf 05 00 00 00      mov    $0x5,%edi
1725:  e8 07 ff ff ff      call   1631 <func4_1>

172a:  39 44 24 0c          cmp    %eax,0xc(%rsp)
172e:  75 64                jne    1794 <phase_4+0xa4>

1730:  48 8d 7c 24 10       lea    0x10(%rsp),%rdi
1735:  e8 2d 05 00 00       call   1c67 <string_length>
173a:  83 f8 02             cmp    $0x2,%eax
173d:  75 5c                jne    179b <phase_4+0xab>
173f:  48 8d 5c 24 14       lea    0x14(%rsp),%rbx
1744:  49 89 d9             mov    %rbx,%r9
1747:  41 b8 42 00 00 00     mov    $0x42,%r8d
174d:  b9 43 00 00 00       mov    $0x43,%ecx
1752:  ba 41 00 00 00       mov    $0x41,%edx
1757:  be 1d 00 00 00       mov    $0x1d,%esi
175c:  bf 05 00 00 00       mov    $0x5,%edi
1761:  e8 f1 fe ff ff      call   1657 <func4_2>

1766:  48 8d 7c 24 10       lea    0x10(%rsp),%rdi
176b:  48 89 de             mov    %rbx,%rsi
176e:  e8 11 05 00 00       call   1c84 <strings_not_equal>
1773:  85 c0               test   %eax,%eax
1775:  75 2b                jne    17a2 <phase_4+0xb2>
1777:  48 8b 44 24 18       mov    0x18(%rsp),%rax
177c:  64 48 2b 04 25 28 00 sub    %fs:0x28,%rax
1783:  00 00
1785:  75 22                jne    17a9 <phase_4+0xb9>
1787:  48 83 c4 20          add    $0x20,%rsp
178b:  5b                  pop    %rbx
178c:  c3                  ret
```

可以看出，程序首先使用 `sscanf` 函数与模式字符串 `"%d %2s"` 从输入中读取了一个整数和一个字符串。

```
mov    $0x5,%edi
call   1631 <func4_1>
```

然后，程序以 `func4_1(5)` 调用了函数 `func4_1` 来计算一个整数值，并将结果与输入的整数进行比较。如果不匹配则引爆炸弹。

接着，程序检查输入字符串的长度是否为 2。如果不是，则引爆炸弹。

```
mov    $0x42,%r8d
mov    $0x43,%ecx
mov    $0x41,%edx
mov    $0x1d,%esi
mov    $0x5,%edi
call   1657 <func4_2>
```

然后，程序以`func4_2(5, 29, 'A', 'C', 'B')`调用了函数 `func4_2` 来生成一个字符串，并将生成的字符串与输入字符串进行比较。如果不匹配则引爆炸弹。

通过分析递归函数 `func4_1` 和 `func4_2` 的汇编代码，可以写出它们的伪代码如下：

```
int func4_1(int n) {
    if (n <= 0) return 0;
    if (n == 1) return n;
    return 2 * func4_1(n-1) + 1;
}
```

```
void func4_2(int n, int target, char char1, char char2, char char3, char* result) {
    if (n == 1) {
        result[0] = char1; // 第一个字符
        result[1] = char2; // 第二个字符
        result[2] = '\0';  // 字符串结束符
        return;
    }

    int prev_value = func4_1(n-1);

    if (target == prev_value + 1) {
        result[0] = char1;
        result[1] = char2;
        result[2] = '\0';
    }
    else if (target >= prev_value) {
        // 递归, 轮换字符顺序: char3, char1, char2
        func4_2(n-1, target, char3, char1, char2, result);
    }
    else {
        // 递归, 调整target并轮换字符顺序: char2, char3, char1
        func4_2(n-1, target - prev_value - 1, char2, char3, char1, result);
    }
}
```

通过模拟输入，我们得到`phase_4`的正确输入为：

31 BA

Phase 5：数组移动累加和

通过阅读汇编代码

```
00000000000017ae <phase_5>:
17d1:  e8 7a f9 ff ff      call 1150 <__isoc99_sscanf@plt>
17d6:  83 f8 01            cmp    $0x1,%eax
17d9:  7e 06              jle    17e1 <phase_5+0x33>
17db:  83 3c 24 00        cmpl   $0x0,(%rsp)
```

```

17df:  78 05                js      17e6 <phase_5+0x38>
17e1:  e8 03 07 00 00      call   1ee9 <explode_bomb>

17e6:  8b 04 24            mov     (%rsp),%eax
17e9:  83 e0 0f            and     $0xf,%eax
17ec:  89 04 24            mov     %eax,(%rsp)
17ef:  83 f8 0f            cmp     $0xf,%eax
17f2:  74 32              je      1826 <phase_5+0x78>
17f4:  b9 00 00 00 00      mov     $0x0,%ecx
17f9:  ba 00 00 00 00      mov     $0x0,%edx
17fe:  48 8d 35 5b 1a 00 00 lea     0x1a5b(%rip),%rsi      # 3260 <array.0>

1805:  83 c2 01            add     $0x1,%edx
1808:  48 98              cltq
180a:  8b 04 86            mov     (%rsi,%rax,4),%eax
180d:  01 c1              add     %eax,%ecx
180f:  83 f8 0f            cmp     $0xf,%eax
1812:  75 f1              jne     1805 <phase_5+0x57>

1814:  c7 04 24 0f 00 00 00 movl    $0xf,(%rsp)
181b:  83 fa 03            cmp     $0x3,%edx
181e:  75 06              jne     1826 <phase_5+0x78>
1820:  39 4c 24 04          cmp     %ecx,0x4(%rsp)
1824:  74 05              je      182b <phase_5+0x7d>
1826:  e8 be 06 00 00      call   1ee9 <explode_bomb>

182b:  48 8b 44 24 08      mov     0x8(%rsp),%rax
...
183f:  c3                ret

```

可知道，程序首先使用 `sscanf` 函数从输入中读取了两个数字，如果读取结果 ≤ 1 或者第一个数字 ≥ 0 ，则引爆炸弹。

接着，程序对第一个数字进行了按位与操作，保留了其低 4 位的值（即对 16 取模）。然后，程序检查该值是否等于 15，如果是，则引爆炸弹。

然后，程序进入一个循环，从一个预设的数组 `array.0` 中读取与该值对应的元素，并将这些元素累加起来。循环继续，直到读取到的元素等于 15 为止。

使用gdb指令

```

(gdb) x/16wd 0x0000555555557260
0x555555557260 <array.0>:      10      2      14      7
0x555555557270 <array.0+16>:    8       12     15     11
0x555555557280 <array.0+32>:    0        4      1     13
0x555555557290 <array.0+48>:    3        9      6      5

```

可以看到数组 `array.0` 的内容。

通过后续汇编代码，我们发现

```

181b:  83 fa 03            cmp     $0x3,%edx
181e:  75 06              jne     1826 <phase_5+0x78>
1820:  39 4c 24 04          cmp     %ecx,0x4(%rsp)
1824:  74 05              je      182b <phase_5+0x7d>

```

意味着循环结束时，循环次数必须为 3，并且累加和必须等于输入的第二个数字。

从15在数组中的位置我们可以倒推

15 <- 6 <- 14

而14 = array.0[2], 又由于 -14 & 0xf = 2

因此, Phase 5 的正确输入为:

-14 35

Phase 6: 链表结点重排

通过阅读汇编代码

```
0000000000001845 <phase_6>:
1867: e8 3d 07 00 00      call 1fa9 <read_six_numbers>
186c: 41 be 01 00 00 00   mov $0x1,%r14d
1872: 49 89 e4            mov %rsp,%r12
1875: eb 28              jmp 189f <phase_6+0x5a>
1877: e8 6d 06 00 00     call 1ee9 <explode_bomb>
187c: eb 30              jmp 18ae <phase_6+0x69>

187e: 48 83 c3 01        add $0x1,%rbx
1882: 83 fb 05           cmp $0x5,%ebx
1885: 7f 10             jg 1897 <phase_6+0x52>
1887: 41 8b 04 9c        mov (%r12,%rbx,4),%eax
188b: 39 45 00           cmp %eax,0x0(%rbp)
188e: 75 ee            jne 187e <phase_6+0x39>
1890: e8 54 06 00 00     call 1ee9 <explode_bomb>
1895: eb e7            jmp 187e <phase_6+0x39>
1897: 49 83 c6 01        add $0x1,%r14
189b: 49 83 c5 04        add $0x4,%r13
189f: 4c 89 ed          mov %r13,%rbp
18a2: 41 8b 45 00        mov 0x0(%r13),%eax
18a6: 83 e8 01          sub $0x1,%eax
18a9: 83 f8 05          cmp $0x5,%eax
18ac: 77 c9            ja 1877 <phase_6+0x32>
18ae: 41 83 fe 05        cmp $0x5,%r14d
18b2: 7f 05            jg 18b9 <phase_6+0x74>
18b4: 4c 89 f3          mov %r14,%rbx
18b7: eb ce            jmp 1887 <phase_6+0x42>

18b9: be 00 00 00 00     mov $0x0,%esi
18be: 8b 0c b4          mov (%rsp,%rsi,4),%ecx
18c1: b8 01 00 00 00     mov $0x1,%eax
18c6: 48 8d 15 43 49 00 00 lea 0x4943(%rip),%rdx # 6210 <node1>
18cd: 83 f9 01          cmp $0x1,%ecx
18d0: 7e 0b            jle 18dd <phase_6+0x98>
18d2: 48 8b 52 08        mov 0x8(%rdx),%rdx
18d6: 83 c0 01          add $0x1,%eax
18d9: 39 c8            cmp %ecx,%eax
18db: 75 f5            jne 18d2 <phase_6+0x8d>
18dd: 48 89 54 f4 20     mov %rdx,0x20(%rsp,%rsi,8)
18e2: 48 83 c6 01        add $0x1,%rsi
18e6: 48 83 fe 06        cmp $0x6,%rsi
18ea: 75 d2            jne 18be <phase_6+0x79>
18ec: 48 8b 5c 24 20     mov 0x20(%rsp),%rbx
18f1: 48 8b 44 24 28     mov 0x28(%rsp),%rax
18f6: 48 89 43 08        mov %rax,0x8(%rbx)
18fa: 48 8b 54 24 30     mov 0x30(%rsp),%rdx
18ff: 48 89 50 08        mov %rdx,0x8(%rax)
1903: 48 8b 44 24 38     mov 0x38(%rsp),%rax
1908: 48 89 42 08        mov %rax,0x8(%rdx)
190c: 48 8b 54 24 40     mov 0x40(%rsp),%rdx
```

```
1911:  48 89 50 08      mov    %rdx,0x8(%rax)
1915:  48 8b 44 24 48    mov    0x48(%rsp),%rax
191a:  48 89 42 08      mov    %rax,0x8(%rdx)
191e:  48 c7 40 08 00 00 00  movq   $0x0,0x8(%rax)
1925:  00
1926:  bd 05 00 00 00    mov    $0x5,%ebp
192b:  eb 09           jmp    1936 <phase_6+0xf1>
192d:  48 8b 5b 08      mov    0x8(%rbx),%rbx
1931:  83 ed 01         sub    $0x1,%ebp
1934:  74 11           je     1947 <phase_6+0x102>
1936:  48 8b 43 08      mov    0x8(%rbx),%rax
193a:  8b 00           mov    (%rax),%eax
193c:  39 03           cmp    %eax,(%rbx)
193e:  7d ed           jge    192d <phase_6+0xe8>
1940:  e8 a4 05 00 00    call   1ee9 <explode_bomb>
1945:  eb e6           jmp    192d <phase_6+0xe8>

1947:  48 8b 44 24 58    mov    0x58(%rsp),%rax
...
1963:  c3             ret
```

我们清晰地看到，程序首先使用 `read_six_numbers` 函数从输入中读取了六个数字，并将它们存储在栈上。然后，程序对这六个数字进行了验证，确保它们在 1 到 6 之间且没有重复。如果验证失败，则引爆炸弹。

接着，程序根据输入的六个数字重新排列了一个预设的链表。每个数字对应链表中的一个结点，程序按照输入的顺序将这些结点连接起来，形成一个新的链表。最后，程序检查新链表中的结点值是否按降序排列。如果不是，则引爆炸弹。

使用gdb指令

```
(gdb) x/24wd 0x000055555555a210
0x55555555a210 <node1>: 375      1      1431675424      21845
0x55555555a220 <node2>: 325      2      1431675440      21845
0x55555555a230 <node3>: 643      3      1431675456      21845
0x55555555a240 <node4>: 550      4      1431675472      21845
0x55555555a250 <node5>: 481      5      1431675232      21845
```

我们发现链表不是连续存储的，但是通过分析他们存储的地址，我们可以找到<node5>的地址

```
1431675424 -> node2 0x55555555a220
1431675440 -> node3 0x55555555a230
1431675456 -> node4 0x55555555a240
1431675472 -> node5 0x55555555a250
1431675232 -> node6 0x55555555a160
```

再通过gdb指令找到节点<node6>的值

```
0x55555555a160 <node6>: 270      6      0      0
```

通过重排节点值使其按降序排列，我们可以得到正确的输入顺序为：

3 4 5 1 2 6

因此，Phase 6 的正确输入为：

3 4 5 1 2 6

Secret Phase：寻找任务开始方式

通过阅读汇编代码

```

0000000000002124 <phase_defused>:
 2124: 48 83 ec 08      sub    $0x8,%rsp
 2128: bf 01 00 00 00   mov    $0x1,%edi
 212d: e8 ea fc ff ff   call   1e1c <send_msg>
 2132: 83 3d df 45 00 00 06  cmpl   $0x6,0x45df(%rip)      # 6718 <num_input_strings>
 2139: 74 05            je     2140 <phase_defused+0x1c>
 213b: 48 83 c4 08      add    $0x8,%rsp
 213f: c3              ret

 2140: 0f b6 0d 31 48 00 00  movzbl 0x4831(%rip),%ecx      # 6978 <input_strings+0x258>
 2147: 84 c9            test   %cl,%cl
 2149: 74 34            je     217f <phase_defused+0x5b>
 214b: b8 01 00 00 00   mov    $0x1,%eax
 2150: ba 00 00 00 00   mov    $0x0,%edx
 2155: 48 8d 3d 1c 48 00 00  lea     0x481c(%rip),%rdi      # 6978 <input_strings+0x258>

 215c: 80 f9 20         cmp    $0x20,%cl
 215f: 0f 94 c1         sete   %cl
 2162: 0f b6 c9         movzbl %cl,%ecx
 2165: 01 ca           add    %ecx,%edx
 2167: 89 c6           mov    %eax,%esi
 2169: 0f b6 0c 07      movzbl (%rdi,%rax,1),%ecx
 216d: 48 83 c0 01      add    $0x1,%rax
 2171: 83 fa 05         cmp    $0x5,%edx
 2174: 7f 04           jg     217a <phase_defused+0x56>
 2176: 84 c9           test   %cl,%cl
 2178: 75 e2           jne    215c <phase_defused+0x38>
 217a: 83 fa 06         cmp    $0x6,%edx
 217d: 74 1a           je     2199 <phase_defused+0x75>

 217f: 48 8d 3d 22 12 00 00  lea     0x1222(%rip),%rdi      # 33a8 <array.0+0x148>
 2186: e8 e5 ee ff ff   call   1070 <puts@plt>
 218b: 48 8d 3d 46 12 00 00  lea     0x1246(%rip),%rdi      # 33d8 <array.0+0x178>
 2192: e8 d9 ee ff ff   call   1070 <puts@plt>
 2197: eb a2           jmp     213b <phase_defused+0x17>

 2199: 48 63 f6         movslq %esi,%rsi
 219c: 48 8d 05 d5 47 00 00  lea     0x47d5(%rip),%rax      # 6978 <input_strings+0x258>
 21a3: 48 8d 3c 06       lea     (%rsi,%rax,1),%rdi
 21a7: 48 8d 35 93 14 00 00  lea     0x1493(%rip),%rsi      # 3641 <array.0+0x3e1>
 21ae: e8 d1 fa ff ff   call   1c84 <strings_not_equal>
 21b3: 85 c0           test   %eax,%eax
 21b5: 75 c8           jne    217f <phase_defused+0x5b>

 21b7: 48 8d 3d 8a 11 00 00  lea     0x118a(%rip),%rdi      # 3348 <array.0+0xe8>
 21be: e8 ad ee ff ff   call   1070 <puts@plt>
 21c3: 48 8d 3d a6 11 00 00  lea     0x11a6(%rip),%rdi      # 3370 <array.0+0x110>
 21ca: e8 a1 ee ff ff   call   1070 <puts@plt>

 21cf: b8 00 00 00 00   mov    $0x0,%eax
 21d4: e8 b7 f9 ff ff   call   1b90 <secret_phase>
 21d9: eb a4           jmp     217f <phase_defused+0x5b>

```

我们找到了调用secret_phase函数的入口点。

想要到达secret_phase函数，必须满足以下条件：

- 已经输入了6个字符串 (`num_input_strings == 6`)
- # 6978 `<input_strings+0x258>`处输入包含六个空格字符 (ASCII 0x20)
- 读取完六个空格之后的字符串与# 3641 `<array.0+0x3e1>`处的字符串相同

首先我们通过输入完成后使用gdb指令查找`<input_strings>`中的存储方式

```
(gdb) x/2s 0x000055555555a720
0x55555555a720 <input_strings>: "Abidel Rid wezha ray frockeme zaridew encor ed xas. An
betdariuzis konsis dlle."
0x55555555a770 <input_strings+80>: ""

(gdb) x/2s 0x000055555555a798
0x55555555a798 <input_strings+120>: "1028424 766476 935124 551926"
0x55555555a7b5 <input_strings+149>: ""

(gdb) x/2s 0x000055555555a810
0x55555555a810 <input_strings+240>: "5 -721"
0x55555555a817 <input_strings+247>: ""

(gdb) x/2s 0x000055555555a888
0x55555555a888 <input_strings+360>: "31 BA"
0x55555555a88e <input_strings+366>: ""

(gdb) x/2s 0x000055555555a900
0x55555555a900 <input_strings+480>: "-14 35"
0x55555555a907 <input_strings+487>: ""
```

可以看到`<input_strings+0x258>`即`<input_strings+600>`对应的是phase_6的输入

接着我们查看# 3641 `<array.0+0x3e1>`处的字符串内容

```
(gdb) x/s 0x555555557641
0x555555557641: "mystic"
```

因此我们只需要将phase_6的输入修改为3 4 5 1 2 6 mystic即可成功进入secret_phase函数。

修改完后`<input_strings+0x258>`处的内容为

```
(gdb) x/2s 0x000055555555a978
0x55555555a978 <input_strings+600>: "3 4 5 1 2 6 mystic"
0x55555555a98b <input_strings+619>: ""
```

Phase 7: 骑士移动 (类似象棋"马"走法)

通过阅读汇编代码

```
000000000001b90 <secret_phase>:
 1b90: 53                push    %rbx
 1b91: 48 8d 3d 74 16 00 00 lea     0x1674(%rip),%rdi    # 320c <_IO_stdin_used+0x20c>
 1b98: e8 d3 f4 ff ff    call   1070 <puts@plt>

 1b9d: e8 48 04 00 00    call   1fea <read_line>
 1ba2: 48 89 c3          mov     %rax,%rbx
 1ba5: 48 89 c7          mov     %rax,%rdi
 1ba8: e8 ba 00 00 00    call   1c67 <string_length>
```

```
1bad: 83 f8 14      cmp    $0x14,%eax
1bb0: 7f 2e          jg     1be0 <secret_phase+0x50>

1bb2: b9 00 00 00 00  mov    $0x0,%ecx
1bb7: ba 00 00 00 00  mov    $0x0,%edx
1bbc: be 00 00 00 00  mov    $0x0,%esi
1bc1: 48 89 df        mov    %rbx,%rdi
1bc4: e8 a0 fd ff ff  call   1969 <func7>
1bc9: 85 c0          test   %eax,%eax
1bcb: 74 1a          je     1be7 <secret_phase+0x57>

1bcd: 48 8d 3d fc 15 00 00  lea     0x15fc(%rip),%rdi      # 31d0 <_IO_stdin_used+0x1d0>
1bd4: e8 97 f4 ff ff  call   1070 <puts@plt>
1bd9: e8 46 05 00 00  call   2124 <phase_defused>
1bde: 5b            pop     %rbx
1bdf: c3            ret

1be0: e8 04 03 00 00  call   1ee9 <explode_bomb>
1be5: eb cb          jmp     1bb2 <secret_phase+0x22>
1be7: e8 fd 02 00 00  call   1ee9 <explode_bomb>
1bec: eb df          jmp     1bcd <secret_phase+0x3d>
```

我们发现程序调用 `read_line` 函数读取用户输入的字符串，并将其存储在栈上。接着，程序调用 `string_length` 函数计算输入字符串的长度，并检查其是否大于 20。如果是，则引爆炸弹。然后，程序调用 `func7` 函数对输入字符串进行验证。如果 `func7` 返回 1，则表示输入字符串有效，炸弹解除；否则，引爆炸弹。

所以 `secret_phase` 实际是调用 `func7(input_string, 0, 0, 0)` 来判断是否解除炸弹的。

通过阅读 `func7` 的汇编代码，我们发现有一个8x8的棋盘，骑士从初始位置 (0, 0) 出发，目标位置是 (4, 7)。骑士的移动方式类似于中国象棋中的“马”走法，即可以走“日”字形的路径。函数 `func7` 通过输入的字符串决定骑士的移动路径，验证路径是否合法，并验证能否到达目的地。

使用gdb指令找到程序中预设的棋盘位置

```
(gdb) x/128bx 0x000055555555a1a0
0x55555555a1a0 <row0>:  0x00    0x00    0x01    0x00    0x00    0x01    0x00    0x00
0x55555555a1a8 <row0+8>:      0xb0    0xa1    0x55    0x55    0x55    0x55    0x00    0x00

0x55555555a1b0 <row1>:  0x00    0x00    0x00    0x01    0x00    0x00    0x00    0x01
0x55555555a1b8 <row1+8>:      0xc0    0xa1    0x55    0x55    0x55    0x55    0x00    0x00

0x55555555a1c0 <row2>:  0x01    0x00    0x01    0x00    0x00    0x01    0x00    0x00
0x55555555a1c8 <row2+8>:      0xd0    0xa1    0x55    0x55    0x55    0x55    0x00    0x00

0x55555555a1d0 <row3>:  0x01    0x00    0x00    0x00    0x00    0x00    0x00    0x00
0x55555555a1d8 <row3+8>:      0xe0    0xa1    0x55    0x55    0x55    0x55    0x00    0x00

0x55555555a1e0 <row4>:  0x00    0x01    0x00    0x00    0x01    0x00    0x01    0x00
0x55555555a1e8 <row4+8>:      0xf0    0xa1    0x55    0x55    0x55    0x55    0x00    0x00

0x55555555a1f0 <row5>:  0x01    0x00    0x00    0x01    0x01    0x00    0x00    0x00
0x55555555a1f8 <row5+8>:      0x00    0xa2    0x55    0x55    0x55    0x55    0x00    0x00

0x55555555a200 <row6>:  0x00    0x00    0x00    0x00    0x00    0x01    0x00    0x01
0x55555555a208 <row6+8>:      0x50    0xa1    0x55    0x55    0x55    0x55    0x00    0x00
```

我们发现他不仅存储了棋盘内容，还存储了下一行棋盘的位置

```

0x000055555555a1b0 -> row1
0x000055555555a1c0 -> row2
0x000055555555a1d0 -> row3
0x000055555555a1e0 -> row4
0x000055555555a1f0 -> row5
0x000055555555a200 -> row6
0x000055555555a150 -> row7

```

通过这个我们就能找到最后一行棋盘的位置

```

(gdb) x/16bx 0x000055555555a150
0x55555555a150 <row7>:  0x00    0x01    0x00    0x00    0x00    0x00    0x00    0x00
0x55555555a158 <row7+8>:      0x00    0x00    0x00    0x00    0x00    0x00    0x00    0x00

```

最终还原棋盘如下：

```

row0[8] = {0, 0, 1, 0, 0, 1, 0, 0};
row1[8] = {0, 0, 0, 1, 0, 0, 0, 1};
row2[8] = {1, 0, 1, 0, 0, 1, 0, 0};
row3[8] = {1, 0, 0, 0, 0, 0, 0, 0};
row4[8] = {0, 1, 0, 0, 1, 0, 1, 0};
row5[8] = {1, 0, 0, 1, 1, 0, 0, 0};
row6[8] = {0, 0, 0, 0, 0, 1, 0, 1};
row7[8] = {0, 1, 0, 0, 0, 0, 0, 0};

```

再看func7的汇编代码开头初始化了一个巨大的数组如下：

```

rsp+0x00: array1[8] = [-2, -1, 1, 2, 2, 1, -1, -2]
rsp+0x20: array2[8] = [ 1, 2, 2, 1, -1, -2, -2, -1]
rsp+0x40: array3[8] = [-1, 0, 0, 1, 1, 0, 0, -1]
rsp+0x60: array4[8] = [ 0, 1, 1, 0, 0, -1, -1, 0]

```

通过观察不难发现，骑士的移动依赖于array1和array2，而array3和array4则用于判定中国象棋中“马”的“蹩脚”走法是否合法。

以下是func7的伪代码实现：

```

// 给定的数组
uint8_t row0[8] = {0, 0, 1, 0, 0, 1, 0, 0};
uint8_t row1[8] = {0, 0, 0, 1, 0, 0, 0, 1};
uint8_t row2[8] = {1, 0, 1, 0, 0, 1, 0, 0};
uint8_t row3[8] = {1, 0, 0, 0, 0, 0, 0, 0};
uint8_t row4[8] = {0, 1, 0, 0, 1, 0, 1, 0};
uint8_t row5[8] = {1, 0, 0, 1, 1, 0, 0, 0};
uint8_t row6[8] = {0, 0, 0, 0, 0, 1, 0, 1};
uint8_t row7[8] = {0, 1, 0, 0, 0, 0, 0, 0};

// rows 是一个指针数组，每个元素指向一行
uint8_t* rows[8] = {row0, row1, row2, row3, row4, row5, row6, row7};

// 偏移数组 (从栈初始化中提取)
int offsets[32] = {
    // dx1 数组 [0-7]
    -2, -1, 1, 2, 2, 1, -1, -2,
    // dy1 数组 [8-15]

```

```

1, 2, 2, 1, -1, -2, -2, -1,
// dx2 数组 [16-23]
-1, 0, 0, 1, 1, 0, 0, -1,
// dy2 数组 [24-31]
0, 1, 1, 0, 0, -1, -1, 0
};

// func7 实现
int func7(uint8_t* input, int x, int y, int k) {
    // 检查是否到达目标位置 (4,7)
    if (x == 4 && y == 7) {
        // 到达目标位置, 检查当前字符
        if (k > 0x13) return 0; // k > 19, 返回0
        uint8_t val = input[k];
        if (val == 0) {
            return 1; // 成功条件: 在(4,7)位置且input[k]==0
        }
        // 如果val != 0, 继续处理
    } else {
        // 未到达目标位置
        if (k > 0x13) return 0; // k > 19, 返回0
        uint8_t val = input[k];
        if (val == 0) return 0; // 遇到0但不在目标位置, 返回0
    }

    // 公共处理逻辑
    uint8_t val = input[k];
    int idx = val & 0x7; // 取低3位作为索引

    // 计算第一组新坐标 (r8d, r11d)
    int final_x = x + offsets[idx]; // dx1
    int final_y = y + offsets[idx + 8]; // dy1

    // 检查第一组坐标是否在有效范围内 (0-7)
    if (final_x < 0 || final_x > 7 || final_y < 0 || final_y > 7) {
        return 0; // 坐标超出范围
    }

    // 计算第二组新坐标 (eax, edx)
    int new_x1 = x + offsets[idx + 16]; // dx2
    int new_y1 = y + offsets[idx + 24]; // dy2

    // 检查最终坐标是否在有效范围内 (0-7)
    if (new_x1 < 0 || new_x1 > 7 || new_y1 < 0 || new_y1 > 7) {
        return 0;
    }

    // === 检查网格位置 (是否存在“蹩脚”) ===
    if (rows[new_x1][new_y1] == 1) {
        return 0;
    }

    // === 检查网格位置 (目的地是否有障碍) ===
    if (rows[final_x][final_y] == 1) {
        return 0;
    }

    // === 递归调用 ===
    return func7(input, final_x, final_y, k + 1);
}

```

在程序运行过程中, 我们通过如下的gdb指令监控骑士的移动

```
break explode_bomb
break func7
break *func7+0x1B9
break *func7+0x1E7

Breakpoint 2, 0x0000555555555969 in func7 ()
(gdb) print $esi      # x
(gdb) print $ecx      # y
(gdb) c

Breakpoint 3, 0x0000555555555b22 in func7 ()
(gdb) print $eax      # new_x1
(gdb) print $edx      # new_y1
(gdb) c

Breakpoint 4, 0x0000555555555b50 in func7 ()
(gdb) print $r8d      # final_x
(gdb) print $r11d     # final_y
(gdb) c
```

再通过编写一个暴力DFS脚本（其实可以通过棋盘一步步分析，不过分支可能太多效率低下），我们可以找到一组可行的骑士移动路径，最终成功到达目标位置 (4, 7)。

其对应的字符串为：

ccaac

总结

通过此次BombLab的逆向分析，我们在每个阶段都涉及了不同的逆向技术和逻辑推理，从简单的字符串比较到复杂的数据结构操作，再到模拟骑士在棋盘上的移动。

通过仔细分析汇编代码，理解程序逻辑，并利用gdb进行动态调试，我们能够逐步揭开每个阶段的谜题，最终成功解除炸弹。

本次实验很好的培训了我们阅读汇编代码的能力，增强了我们解决复杂问题的技巧，也加深了我们对计算机底层运行机制的理解。

同时也教会了我们使用gdb等调试工具的重要性，这些工具在逆向工程中是不可或缺的利器。